Part B

Part B: how far is today's hardware from factoring RSA-2048?

Short answer. Published estimates need 10^5 to 10^6 good physical qubits (Gidney 2025: 897,864; Pinnacle 2026: 94,000 to 381,000). The lowest verified two-qubit error on about 100 qubits is Quantinuum Helios (98 qubits, 7.9e-4); the largest verified array has 6,100 atoms (count only). The qubit gap is about 150x to 10^4x, plus an error-correction gap and, for slow-clock machines, a speed gap. It is not "now" and not "never": the requirement fell about 10,000x in 14 years. Every number is printed by `part_b.py`.

1. Published estimates, and why qubits trade against runtime

Source	Target	Physical qubits	Runtime	Key assumptions
Gidney, Ekera 2019, 1905.09749, Table 2, 28-factory row	RSA-2048	20 M	0.31 d (7.4 h)	0.1% gate error, 1 us cycle, 10 us reaction, 2D nearest-neighbour grid
same table, 14 factories / 1 factory	RSA-2048	19 M / 16 M	0.36 d / 6 d	same
Gidney 2025, 2505.15917	RSA-2048	897,864 ("<1 M")	4.96 d ("<1 week")	same as 2019
Webster et al. (Iceberg) 2026, 2602.11457 v2, Table VI	RSA-2048	94 k / 135 k / 381 k	month / week / day	qLDPC codes, 0.1%, 1 us; needs non-local connectivity; simulation only
Babbush et al. (Google Quantum AI) 2026, 2603.28846	ECC-256, not RSA	<500 k	18-23 min; 9-12 min "primed"	superconducting, planar; circuits withheld (zero-knowledge proof)

One correction to the brief: Pinnacle does state runtimes (its "<100,000" is for about a month). Clarification: Google's 9-12 minutes is the "primed" time for elliptic curves, not RSA.

Qubits against time. A run is a fixed pile of logical operations (6.5e9 Toffolis in 2025) and magic-state factories supply the Toffolis. More factories: faster, more qubits. Fewer: slower, fewer qubits. Own arithmetic: 2019 to 2025 cut qubits 20e6 / 897,864 = **22.3x** and lengthened the run 4.96 / 0.31 = **16x**. Spacetime volume fell only from 5.9 to 4.45 megaqubit-days, **1.33x**. So most of the "20x" is time bought with qubits, and the assumptions were unchanged (algorithms, not hardware). The real volume savers: approximate residue arithmetic (1,399 logical qubits instead of 6,189), yoked surface codes (430 physical per idle logical qubit instead of 1,352) and magic state cultivation (six small factories). Pinnacle shows the knob: 94 k qubits for a month, 381 k for a day.

2. Toy surface-code model (own calculation)

p_L = 0.1 (p / p_th)^((d+1)/2), p_th = 1%, 2 d^2 physical per logical, choose the smallest odd d with L T p_L < 0.1 (T = cycles).

Judges' example: L = 1,500, 5 days at 1 us: T = 5 x 86,400 / 1e-6 = 4.32e11. L T = 6.48e14, so p_L < 0.1 / 6.48e14 = 1.54e-16. At p = 0.1%, p_L = 0.1 x 0.1^((d+1)/2):

d	25	27	29	31
p_L	1e-14	1e-15	1e-16	1e-17
L T p_L	6.48	0.648	**0.0648**	0.0065

d = 29, 2 x 29^2 = **1,682** physical per logical, 1,500 x 1,682 = **2.52 million** qubits before factories and routing. Checks: p = 0.2% gives 5.55 M, p = 0.5% gives 29 M; 8 hours gives 2.19 M, one year 3.27 M (runtime enters only through log(L T)).

On Gidney's inputs (L = 1,399, 4.96 d) the toy gives d = 29 and 2.35 M, 2.6x his 897,864: it is 10x more pessimistic per patch and has no cold storage or yoking. The order of magnitude (10^6 to 10^7) is reproduced.

3. Today's machines against the requirement

Checked: Quantinuum Helios, 2511.05465: 98 qubits, two-qubit infidelity 7.9e-4. Google Willow, 2408.13687: distance-7 memory on 101 qubits, Lambda = 2.14, 0.143% logical error per 1.1 us cycle, decoder latency 63 us. Largest array: 2403.12021 (Nature 2025), 6,100 trapped atoms, coherence only, no gate error at that scale. Not verified: IBM Condor, 1,121 qubits (2023).

- **Qubits:** 897,864 / 98 = **9,200x**; toy 2.52 M / 98 = 26,000x; Pinnacle 100 k / 98 = 1,000x. Against the largest array: 897,864 / 6,100 = **147x** (toy 414x, Pinnacle 16x).
- **Gate error:** 7.9e-4 already beats the assumed 1e-3, but on 98 qubits, not 10^6. On small devices scale is the gap.
- **Error correction:** the toy needs p_L = 1.5e-16 per logical qubit per cycle; the best measured memory is 1.4e-3 (gap 9e12). With Willow's Lambda = 2.14 instead of 10 the toy needs d = 87 and 22.7 M qubits (9x more). Logical qubits: 1,399 needed, one memory shown. Decoder: 63 us against an assumed 10 us reaction time (6x).
- **Clock:** atom and ion machines are slow-clock (the distinction is drawn in 2603.28846). At an illustrative 1 ms cycle, 1000x the assumed 1 us, 4.96 days become 13.6 years.
- These gaps are coupled through p / p_th; they must not be multiplied.

4. Timeline and consequence (our assumptions, not forecasts)

Fowler et al. 2012 (Table 2 of 1905.09749) needed 1,000 M qubits; 94 k in 2026 is 10,600x fewer in 14 years. Assumptions: A1 hardware keeps 0.1% error and 1 us cycles while scaling (optimistic); A2 qubit count doubles every D years, starting from 98 (or about 1,100, unverified); A3 the requirement keeps halving every H years (H = inf: no more algorithm gains; H = 3 y: slower than the past). Gap 9,162x = 13.2 doublings; years to close = 13.2 / (1/D + 1/H).

Year the gap closes	H = inf, from 98	H = 3 y, from 98	H = inf, from 1,121	H = 3 y, from 1,121
D = 1 y	2039	2036	2036	2033
D = 2 y	2052	2042	2045	2038
D = 3 y	2065	2046	2055	2040

Range 2033 to 2065. Not this decade; plausible within the secrecy lifetime of data encrypted today.

Consequence: harvest now, decrypt later. An adversary can record encrypted traffic today and decrypt it when a machine exists. Mosca: act now if X + Y > Z (X = years the data must stay secret, Y = years to migrate, Z = years to a machine). With our assumed X = 10, Y = 7, any machine before 2043 exposes it: 7 of our 12 scenarios. Key exchange is urgent because recorded sessions cannot be re-protected later: deploy ML-KEM (NIST FIPS 203, hybrid at first) now, inventory where RSA/ECC keys live, migrate signatures (ML-DSA, FIPS 204) after. Google's 2026 paper itself urges migration "without delay" (abstract, 2603.28846). NIST standards are not on arXiv and were not fetched.

Limits. The toy ignores factories, routing and correlated errors; Section 4 is a what-if. Drafted with an AI assistant; numbers verified by running `part_b.py` and reading the cited papers.